Bitonic Sort on GPU: Implementation and Optimization

sbhatnagar45@gatech.edu

I. INTRODUCTION

The objective of this project was to implement an efficient parallel integer sorting algorithm using CUDA, focusing on Bitonic Sort, and to optimize its performance for execution on an NVIDIA H100 GPU. The assignment required analysis using NSight Compute to maximize metrics such as memory throughput, achieved occupancy, and overall speedup. This report discusses the implementation, experiments conducted, performance optimizations (both successful and unsuccessful), and presents a performance analysis based on profiling data.

II. IMPLEMENTATION OVERVIEW

The core of the implementation was based on Bitonic Sort, which is well-suited to GPU architectures due to its predictable parallel pattern. The provided framework (kernel.cu) was extended to implement Bitonic Sort. The basic approach involved:

- **Parallel Kernel Launches:** The Bitonic Sort was mapped to a series of CUDA kernel launches, each corresponding to a specific stage in the sorting network. These kernels perform parallel compare-and-exchange operations on the relevant data segments.
- **Shared Memory Optimization:** Within each thread block, shared memory was used to accelerate sorting on data chunks and minimize global memory traffic, significantly boosting performance.
- **Validation at Scale:** Correctness and performance were rigorously validated using arrays of up to 100 million elements, following the requirements of the provided grading scripts.

A. Optimizations

Several optimization techniques were investigated to improve both memory throughput and occupancy:

1. Naive Global Memory Bitonic Sort (Baseline)
    - Each compare-exchange was performed directly in global memory.
    - Each stage of the sort resulted in a new kernel launch.
2. Shared Memory Tiling
    - Sub-sequences of the array that fit in shared memory were loaded for local sorting.
    - Bitonic sort stages for these blocks were executed in-place within shared memory, greatly reducing global memory accesses.
    - After sorting, the blocks were written back to global memory.
3. Minimizing Kernel Launches
    - Combined multiple sort stages into a single kernel when possible, especially within shared memory tiles.
4. Occupancy and Resource Balancing
    - Experimented with block and grid sizes to maximize SM occupancy.
    - Tuned shared memory usage to avoid register spilling and maximize concurrent thread blocks per SM.
5. Unsuccessful: Warp Shuffling
    - Tried warp shuffle instructions to further reduce shared memory bandwidth, but saw little gain due to the non-contiguous access patterns of Bitonic Sort.
6. Low Impact:Asynchronous Transfers and Managed Memory
    - Used cudaMemcpyAsync for memory copies, but without effective overlap with kernel execution, there was limited benefit.
    - Switched device buffers to cudaMallocManaged, but this did not yield significant performance gains and sometimes introduced additional overhead due to managed memory migration.

III. PERFORMANCE EVALUATON

A. Test Environment & Metrics

All the experiments were run NVIDIA H100 GPU with input array size of 100M. The performance was measured based on the results from the grading script (grade.py). The metrics collected were –

1. **Achieved Occupancy (%):** Maximum percentage of active warps per SM.
2. **Memory Throughput (% of peak):** How efficiently global memory bandwidth is used.
3. **Million Elements Per Second (MEPS):** Throughput of the sorting algorithm.
4. **Kernel Time (ms):** Execution time for the sorting kernels.
5. **D2H and H2D transfer times (ms):** Time to transfer data from device-to-host and host-to-device.

B. Results Summary (on H100)

The results of the experimenation are in the table below. The baseline configuration, which sorts entirely in global memory, is fundamentally limited by high-latency and uncoalesced memory accesses. As a result, this version achieves only moderate occupancy (62.34%), a relatively low throughput (182.17 MEPS), and long kernel execution times (137 ms), although it still manages a significant speedup over CPU sorting (33X). Introducing shared memory for block-wise sorting leads to a dramatic reduction in kernel execution time (from 137 ms down to 3.17 ms), as threads within each block can cooperate and benefit from the much lower latency of shared memory compared to global memory. This optimization also produces a slight improvement in occupancy (from 62.34% to 65%) and throughput (184.48 MEPS), but the main advantage is the sharp decrease in kernel runtime. Notably, memory throughput as a percentage of peak remains almost unchanged, indicating that after the shared memory optimization, the algorithm is no longer strictly bottlenecked by global memory bandwidth. A far more substantial performance gain is realized through data type optimization: converting from 32-bit integers to 16-bit shorts. This reduces the memory footprint per element, enabling each block to process twice as many elements in the same shared memory space and allowing more thread blocks to be scheduled simultaneously on each SM. The result is a dramatic increase in occupancy (jumping to 82.18%), and throughput more than doubles (394.46 MEPS), with the speedup over the CPU soaring to 63X. Although kernel execution time slightly increases compared to the shared memory only version (from 3.17 ms to 5.35 ms), the total throughput and efficiency are greatly enhanced due to better hardware utilization. Interestingly, memory throughput as a percentage of peak bandwidth drops (to 43.75%), which can be explained by the algorithm transitioning from being memory-bound to being limited by computation and shared memory resources. As a result, global memory bandwidth is no longer the primary bottleneck after these optimizations. The choice of MAX_CHUNK_SIZE (1024) also plays a critical role in tuning performance. Reducing the chunk size lowers the shared memory required per block and, in theory, allows higher occupancy by enabling more blocks to run concurrently. However, making the chunk size too small actually led to a decrease in occupancy in this implementation (for instance, from 82.18% to 65%) and diminished overall throughput. This outcome reflects the importance of balancing per-block work with device-wide concurrency—when each block processes too little data, the benefits of shared memory are lost and hardware resources are not fully leveraged.

Further optimizations such as kernel fusion, warp shuffle instructions, and efforts to reduce branch divergence were explored but yielded little additional benefit, as the Bitonic sort's structure inherently constrains the effectiveness of these strategies. Ultimately, the most impactful improvements came from shared memory usage and, especially, data type reduction, which together addressed the principal bottlenecks of the baseline approach and maximized sorting efficiency on the GPU.

Optimization	Occupancy	Memory Throughput	MEPS	Kernel Time (ms)	D2H+H2D (ms)	Speed up	Score
Baseline (Global Memory)	62.34	51.86	182.174	137	419	33X	12.13
Shared Memory	65.0	52.08	184.481	3.17	539	29X	17.22
Datatype Optimization	82.18	43.75	394.46	5.35	248	63X	17.49

IV. ANALYSIS OF OPTMIZATIONS

What Helped

1. **Shared Memory Usage:** Employing shared memory in the bitonic sort kernel resulted in significant performance gains. Loading array segments into shared memory minimized global memory accesses and enabled faster in-block sorting due to lower memory latency. This approach facilitated efficient intra-block data exchanges and relieved pressure on global memory bandwidth.
2. **Pinned (Page-Locked) Host Memory:** Registering host memory with cudaHostRegister allowed Host-to-Device (H2D) and Device-to-Host (D2H) transfers to use direct memory access (DMA), reducing transfer latency and increasing bandwidth.
3. **Asynchronous Memory Transfers:** Using cudaMemcpyAsync enabled some overlap between memory transfers and kernel execution. This approach helped to reduce visible transfer latency, although true overlap was limited by the sequential structure of the host code.
4. **Efficient Data Types:** Reducing the array element type from int to short improved memory bandwidth, cache usage, and occupancy, provided the value range was within 16 bits.

What Didn't Help

1. **Host-Side Padding with Loops:** Padding arrays on the host with loops or using standard functions proved less efficient than device-side approaches or optimized memory routines.
2. **Excessive cudaDeviceSynchronize:** Frequent use of cudaDeviceSynchronize() after every kernel or memory operation led to serialization of GPU tasks, reducing available parallelism and degrading overall performance. Performance improved when synchronization was minimized.
3. **Pinned Memory Variants:** Comparing cudaHostRegister and cudaMallocHost for host memory pinning did not yield significant differences in transfer speed for this workload.
4. **Unified Memory for All Buffers:** Allocating all arrays with cudaMallocManaged did not provide performance benefits over explicit device or pinned memory and sometimes introduced additional overhead.
5. **Reducing Data Type Size to char:** Attempting to use the char type was not feasible due to the possible value range in the array, which exceeded the maximum storable value in an 8-bit type.
6. **Host Asynchronous Calls and Streams:** Employing cudaMemcpyAsync and CUDA streams, without restructuring the host program for true concurrency, did not result in speedup, as the code flow remained inherently sequential.
7. **Larger Block and Chunk Sizes:** Increasing (or decreasing) block sizes and chunk sizes beyond a certain point (for example, raising MAX_CHUNK_SIZE) did not continue to improve throughput and sometimes even made it worse, likely due to shared memory or register constraints.

V. CONCLUSION

Through a series of optimizations—most notably shared memory tiling, data type reduction from int to short, and careful kernel launch parameter tuning—Bitonic Sort was effectively parallelized and accelerated on the NVIDIA H100 GPU. The final implementation achieved a speedup exceeding 60X over the CPU baseline on arrays of 100 million elements, with occupancy rising to over 82% and throughput approaching 394 million elements per second (MEPS). The switch to 16-bit data types and reducing chunk size had the most significant impact, improving both memory efficiency and computational throughput. Leveraging shared memory for in-block sorting minimized slow global memory accesses, and tuning chunk and block sizes helped maximize occupancy and hardware utilization.

Despite substantial improvements, certain limitations remained and some evaluation thresholds could not be met. While kernel time and occupancy targets were achieved, both memory throughput and overall MEPS fell short of the highest benchmarks established for the H100. This outcome appears to result from several factors: the intrinsic synchronization requirements of Bitonic Sort, limited overlap between memory transfers and kernel execution, and possibly suboptimal parameter or block size choices. Although optimizations such as pinned memory and asynchronous transfers provided some benefit, their impact diminished after initial gains, and further overlap of data transfers with computation proved less effective as bottlenecks shifted to shared memory bandwidth and kernel computation rather than memory throughput and transfer. Further improvements might be possible by restructuring the program for greater concurrency—such as overlapping memory transfers with computation using multiple CUDA streams—or by applying more aggressive parameter tuning. Exploring these approaches could enable better utilization of available bandwidth and hide communication latency more effectively.

Looking ahead, the most promising avenues for closing the remaining performance gap include combining Bitonic Sort with more scalable GPU algorithms (such as merge sort or radix sort) to address the inherent scalability constraints of Bitonic, especially for very large arrays. Additional improvements may be possible by leveraging advanced CUDA features (such as cooperative groups) and further refining kernel launch parameters and occupancy. This project demonstrates that, while data type optimization, shared memory usage, and careful kernel tuning can drive significant speedups for parallel sorting on modern GPUs, maximizing performance requires both deeper concurrency and and an algorithm best matched to the hardware's strengths.